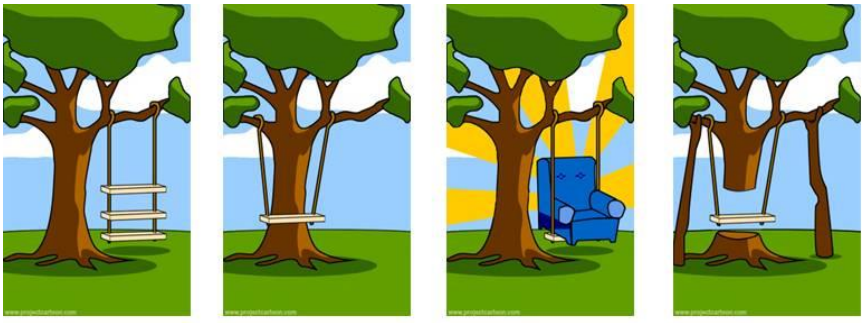
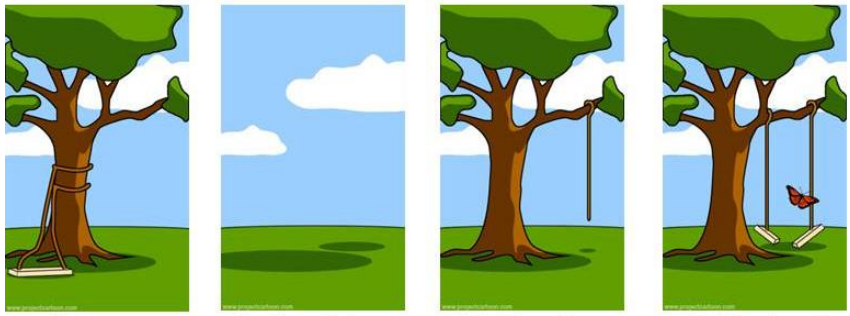
2. Software development
writing simple programs

"Software Development Methodologies"
Ricardo Filipe, ISEC (ricardo.filipe@isec.pt)

SW development - Only a developer problem?



SW development- Only a developer problem?



18

SW development- Only a developer problem?



19

Problem 1

- Given a collection of lines of text (strings) stored in a file, sort them in alphabetical order, and write them into another file.

The situation

- Which tasks are involved in writing this simple program?
- Some important issues:
  1. Requirements clarification
  2. Design constraints and decisions
  3. Effort estimation
  4. Code implementation
  5. Testing

1. Requirements

- Requirements:

  Statements that define and qualify what the program needs to do

  1. Functional requirements

     What the program does

  2. Nonfunctional requirements

     The manner in which the functional requirements need to be achieved

The quicksort algorithm shall be used.

The program shall be able to sort 1000 strings in less than 1 second.

1. Requirements (must be clarified)

- The developer must clarify the requirements with the client
- Example of points to be clarified:
  - Functional requirements
    - Input formats
    - Sorting options
    - Special cases, boundaries, error conditions
  - Nonfunctional requirements
    - Performance
    - Real-time
    - Modifiability

2. Design constraints and decisions

- Design constraints:

  Statements that constrain the ways in which the software can designed and implemented

- Some constraints that must be made:
  - What kind of User Interface?
  - What's the typical and maximum file size?
  - On which platforms will the program run?
- Some decisions:
  - What programming language shall be used?
  - What sorting algorithm?
- Design constraints and decisions may be part of the requirements

24

Architectural design

- Design the program:
  - Divide into basic blocks
  - Ex:
    - StringSorter
    - UserInterface
  - Define the interfaces
    - Public methods
    - Interactions

25

3. Effort Estimation

- Estimate effort or time?
- Estimate task or subtasks?
- Create a work breakdown or product breakdown?
- Which factors to consider?
  - Learning curve, Available Resources, …
- How good are you as an estimator?
  - Do you have historical data?

4. Code implementation

- Use code conventions
  - Choose descriptive names and capitalizations according to the programming language
  - Use rules for indentation, comments, declarations, statements
  - Use good programming practices
- Test
  - Unit test immediately after finishing the unit
- Code review
  - Perform code review
- Use the standard library
  - Don’t reinvent the wheel

5. Testing

- Acceptance testing
  - Planned as soon as the requirements are finished
  - Executed when the program has completed
    - to assure the client the program runs as specified
  - Black-box test
- Unit testing
  - Executed by the programmer
  - Assesses individual units
  - White-box testing
  - Use testing frameworks
- Stress Tests?

Problem 2

- Is developing a payroll system a different problem?

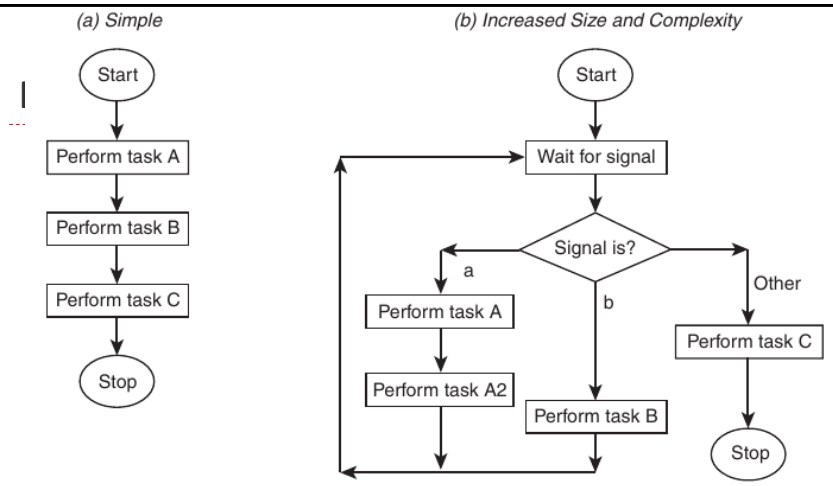


Figure 2.1 Size and complexity.

As shown in Figure 2.1, a relatively minor increase in the number of tasks and decisions has greatly increased the complexity. As is the case for a single programming module, when both size and complexity are magnified several times in a software system problem, the solution to those problems also involves a comparable expansion in size and complexity.

Situations to be addressed

1. Size and complexity
   - Decomposition, modularization, separation, incremental iterations
2. Technology and tools
   - Programming language and development tools
   - Database, network, middleware
3. Process and methodology
   - Choose software development process

   The set of tasks, the sequence and flow of these tasks, the inputs to and the outputs from the tasks, and the preconditions for each of the tasks involved in the production of software
4. Effort estimation and schedule

31

5. Resources assignment (people)
6. Special concerns in the development lifecycle:
   1. Requirements
   2. Design
   3. Implementation (code and unit test)
   4. Integration and test
   5. Release
   6. Support and maintenance
7. Coordination efforts
   1. Process
      - Quality assurance
   2. Product
   3. People

Bibliography

- **Tsui, Frank e Karam, Orlando.** *Essentials of Software Engineering.* Sudbury, MA : Jones and Bartlett Publishers, 2007 (Chapter 1 and 2)